Home Credit Default Risk

Relatório Técnico — Pipeline de Machine Learning

para Predição de Inadimplência

Valvitor Santos

19 de abril de 2026

Resumo. Este relatório descreve o desenvolvimento de um pipeline completo de machine learning para a competição *Home Credit Default Risk* do Kaggle. O problema consiste em prever a probabilidade de inadimplência de clientes de crédito com histórico financeiro limitado. A solução emprega engenharia de features sobre sete tabelas relacionais, modelo LightGBM com otimização de hiperparâmetros via Optuna e validação cruzada estratificada em 5 folds. A métrica de avaliação é a **AUC-ROC**.

Sumário

1 Introdução e Contexto do Problema **2**
1.1 Motivação . . . 2
1.2 Formulação do Problema . . . 2
1.3 Desafios Principais . . . 2

2 Conjunto de Dados **2**
2.1 Estrutura Relacional . . . 2
2.2 Variável Alvo . . . 3

3 Análise Exploratória de Dados **3**
3.1 Anomalia em `DAYS_EMPLOYED` . . . 3
3.2 Features `EXT_SOURCE` . . . 3
3.3 Bloco Imobiliário . . . 4
3.4 Desbalanceamento de Classes . . . 4

4 Engenharia de Features **4**
4.1 Visão Geral . . . 4
4.2 Features da Tabela Principal . . . 4
4.2.1 Features Financeiras . . . 4
4.2.2 Features Temporais . . . 5
4.2.3 Combinações de `EXT_SOURCE` . . . 5
4.3 Agregação das Tabelas Auxiliares . . . 5
4.3.1 Bureau e Bureau Balance . . . 5
4.3.2 Parcelas (`installments_payments`) . . . 6
4.3.3 Cartão de Crédito (`credit_card_balance`) . . . 6
4.3.4 Aplicações Anteriores (`previous_application`) . . . 6

4.4 Processamento Final 6

5 Modelagem **6**
5.1 Escolha do Algoritmo 6
5.2 Otimização de Hiperparâmetros com Optuna 7
5.3 Treinamento Final com 5-Fold CV 7
5.4 Métrica OOF vs. Média dos Folds 8
5.5 Tracking com MLflow 8

6 Boas Práticas e Reprodutibilidade **8**
6.1 Estrutura de Código 8
6.2 Gerenciamento de Memória 8
6.3 Prevenção de Data Leakage 9

7 Conclusão e Próximos Passos **9**
7.1 Conclusão 9
7.2 Próximos Passos 9

A Dependências **9**

1 Introdução e Contexto do Problema

1.1 Motivação

Instituições financeiras enfrentam o desafio de avaliar a capacidade de pagamento de clientes com pouco ou nenhum histórico de crédito formal. A ausência de dados históricos em bureaus de crédito convencionais leva à exclusão financeira de populações que, de fato, são capazes de honrar seus compromissos. O Home Credit, organização especializada em crédito responsável, busca usar fontes de dados alternativas para tornar essa avaliação mais justa e precisa.

1.2 Formulação do Problema

O objetivo é construir um modelo de **classificação binária** que, dado o perfil de um solicitante, estime a probabilidade de inadimplência:

$$\hat{p}_i = P(\text{TARGET}_i = 1 \mid \mathbf{x}_i) \tag{1}$$

onde $\text{TARGET}_i = 1$ indica que o cliente i não pagou o empréstimo, e $\mathbf{x}_i$ é o vetor de features do cliente.

A métrica de avaliação oficial é a **AUC-ROC** (Área sob a Curva ROC), que mede a capacidade do modelo de separar inadimplentes de adimplentes, independentemente do threshold de classificação:

$$\text{AUC} = \int_0^1 \text{TPR}\big(\text{FPR}^{-1}(t)\big)\, dt \tag{2}$$

1.3 Desafios Principais

- **Desbalanceamento de classes:** aproximadamente 8% dos contratos resultam em inadimplência, criando forte viés em favor da classe majoritária.
- **Dados distribuídos em múltiplas tabelas:** informações relevantes estão fragmentadas em 7 tabelas relacionais que precisam ser agregadas ao nível do solicitante.
- **Alta proporção de valores ausentes:** algumas features possuem até 60% de nulos, exigindo tratamento cuidadoso.
- **Engenharia de features crítica:** a performance depende fortemente das features derivadas, especialmente das tabelas auxiliares.

2 Conjunto de Dados

2.1 Estrutura Relacional

Os dados são fornecidos em 7 arquivos CSV com a seguinte hierarquia:

Tabela	Chave	Linhas	Descrição
application_train/test	SK_ID_CURR	307.511 / 48.744	Dados da solicitação atual
bureau	SK_ID_CURR	1.716.428	Créditos em outros bancos
bureau_balance	SK_ID_BUREAU	27.299.925	Histórico mensal do bureau
previous_application	SK_ID_CURR	1.670.214	Solicitações anteriores
POS_CASH_balance	SK_ID_CURR	10.001.358	Saldos POS e empréstimos
installments_payments	SK_ID_CURR	13.605.401	Histórico de parcelas
credit_card_balance	SK_ID_CURR	3.840.312	Saldos de cartão de crédito

Tabela 1: Estrutura do conjunto de dados

A tabela principal é a âncora do modelo; todas as demais são agregadas ao nível `SK_ID_CURR` e unidas via `left join`.

2.2 Variável Alvo

- `TARGET` = 0: cliente pagou normalmente (91,93%)
- `TARGET` = 1: cliente inadimpliu (8,07%)

3 Análise Exploratória de Dados

A EDA foi conduzida no notebook `01_EDA.ipynb` com o objetivo de extrair insights acionáveis — não apenas estatísticas descritivas, mas decisões de pré-processamento e engenharia de features diretamente informadas pelos dados.

3.1 Anomalia em `DAYS_EMPLOYED`

A variável `DAYS_EMPLOYED` representa há quantos dias o cliente está empregado. Porém, 18.148 clientes apresentam o valor **365.243** — um placeholder para "não empregado", equivalente a $\approx$ 1000 anos de emprego.

Decisão: substituir 365.243 por `NaN` e criar uma feature binária `FLAG_EMPLOYED_ANOMALY` para preservar a informação de que o dado estava ausente.

Listing 1: Tratamento da anomalia em DAYS_EMPLOYED

```
df['FLAG_EMPLOYED_ANOMALY'] = (df['DAYS_EMPLOYED'] == 365243).
    astype(np.int8)
df['DAYS_EMPLOYED'] = df['DAYS_EMPLOYED'].replace(365243, np.nan
    )
```

3.2 Features `EXT_SOURCE`

As variáveis `EXT_SOURCE_1`, `EXT_SOURCE_2` e `EXT_SOURCE_3` representam scores externos de crédito provenientes de bureaus terceiros. A EDA revelou que estas são as **features mais correlacionadas com a inadimplência** de todo o dataset, tornando-as candidatas prioritárias para combinações não-lineares.

`EXT_SOURCE_1` possui 56% de valores ausentes, exigindo a criação de uma flag adicional: `FLAG_EXT_SOURCE_1_MISSING`.

3.3 Bloco Imobiliário

Aproximadamente 20 variáveis relacionadas ao imóvel do cliente (`APARTMENTS_AVG`, `BASEMENTAREA_AVG`, etc.) apresentam mais de 60% de nulos e baixa correlação com o target.

Decisão: remover todas as colunas com mais de 60% de valores ausentes, exceto `EXT_SOURCE`.

3.4 Desbalanceamento de Classes

Com 8,07% de inadimplência, modelos não ajustados tendem a classificar quase todos os clientes como adimplentes. **Decisão:** usar `scale_pos_weight` no LightGBM:

$$\texttt{scale_pos_weight} = \frac{N_{\text{negativos}}}{N_{\text{positivos}}} = \frac{283.301}{24.210} \approx 11{,}70 \tag{3}$$

4 Engenharia de Features

4.1 Visão Geral

A engenharia de features foi executada no notebook `02_feature_engineering.ipynb`, transformando as 7 tabelas brutas em um único dataset processado com mais de 1.000 features. O processo segue quatro etapas:

1. Features manuais na tabela principal
2. Agregação do bureau e bureau_balance
3. Agregação das tabelas auxiliares (previous, POS, installments, credit card)
4. Merge final, One-Hot Encoding e alinhamento

4.2 Features da Tabela Principal

4.2.1 Features Financeiras

Razões financeiras que capturam a relação entre crédito, renda e parcelas:

Feature	Fórmula
`CREDIT_INCOME_RATIO`	$\texttt{AMT_CREDIT}/(\texttt{AMT_INCOME_TOTAL} + 1)$
`ANNUITY_INCOME_RATIO`	$\texttt{AMT_ANNUITY}/(\texttt{AMT_INCOME_TOTAL} + 1)$
`ANNUITY_CREDIT_RATIO`	$\texttt{AMT_ANNUITY}/(\texttt{AMT_CREDIT} + 1)$
`INCOME_PER_PERSON`	$\texttt{AMT_INCOME_TOTAL}/(\texttt{CNT_FAM_MEMBERS} + 1)$
`GOODS_CREDIT_RATIO`	$\texttt{AMT_GOODS_PRICE}/(\texttt{AMT_CREDIT} + 1)$

Tabela 2: Features financeiras criadas

O acŕescimo de 1 no denominador evita divisão por zero sem distorcer valores para denominadores grandes.

4.2.2 Features Temporais

- `DAYS_BIRTH_YEARS`: idade em anos (mais interpretável que dias negativos)
- `DAYS_EMPLOYED_YEARS`: tempo de emprego em anos
- `EMPLOYED_TO_BIRTH_RATIO`: fração da vida trabalhada
- `EMPLOYED_TO_AGE_PCT`: proporção tempo de emprego / idade

4.2.3 Combinações de `EXT_SOURCE`

Dado o alto poder preditivo dos scores externos, foram criadas múltiplas combinações:

$$\texttt{EXT_SOURCE_MEAN} = \tfrac{1}{3}(E_1 + E_2 + E_3) \tag{4}$$

$$\texttt{EXT_SOURCE_STD} = \mathrm{std}(E_1, E_2, E_3) \tag{5}$$

$$\texttt{EXT_SOURCE_PROD} = E_1 \cdot E_2 \cdot E_3 \tag{6}$$

$$\texttt{EXT12} = E_1 \cdot E_2, \quad \texttt{EXT13} = E_1 \cdot E_3, \quad \texttt{EXT23} = E_2 \cdot E_3 \tag{7}$$

O produto captura interações não-lineares: um score baixo em qualquer fonte colapsa o produto, sinalizando risco concentrado.

4.3 Agregação das Tabelas Auxiliares

Para cada tabela auxiliar, o processo é:

1. Criar features manuais específicas para o domínio da tabela
2. Aplicar agregações estatísticas para colunas numéricas: `count`, `mean`, `max`, `min`, `sum`
3. Aplicar One-Hot Encoding + agregação para colunas categóricas: `sum`, `mean`
4. Fazer `left join` ao nível `SK_ID_CURR`

Este processo é implementado nas funções reutilizáveis `agg_numeric()` e `agg_categorical()` em `src/features.py`.

4.3.1 Bureau e Bureau Balance

O `bureau_balance` fornece o histórico mensal de cada crédito. As colunas de status foram binarizadas:

Listing 2: Engenharia de features no bureau_balance

```
# C = pago, X = sem divida, 1-5 = meses de atraso crescente
bureau_balance['STATUS_GOOD'] = bureau_balance['STATUS'].isin(['C','X']).astype(np.int8)
bureau_balance['STATUS_LATE'] = bureau_balance['STATUS'].isin(['1','2','3','4','5']).astype(np.int8)
```

4.3.2 Parcelas (`installments_payments`)

- `PAYMENT_DIFF`: valor pago − valor devido (positivo = pagou mais que o mínimo)
- `PAYMENT_RATIO`: razão entre pago e devido
- `DAYS_PAST_DUE`: dias de atraso (zerado para pagamentos antecipados)
- `PAID_ON_TIME`: flag binária de pontualidade

4.3.3 Cartão de Crédito (`credit_card_balance`)

$$\texttt{UTILIZATION} = \frac{\texttt{AMT_BALANCE}}{\texttt{AMT_CREDIT_LIMIT_ACTUAL} + 1} \tag{8}$$

Alta utilização do limite é um forte indicador de estresse financeiro.

4.3.4 Aplicações Anteriores (`previous_application`)

- `APP_CREDIT_RATIO`: razão entre valor solicitado e aprovado — captura se o cliente tipicamente pede mais do que recebe
- `DAYS_DECISION_YEARS`: tempo desde a última decisão de crédito

4.4 Processamento Final

Após o merge de todas as tabelas:

1. Remoção de colunas com **mais de 80% de nulos** pós-merge
2. **One-Hot Encoding** para variáveis categóricas com menos de 10 categorias únicas
3. **Alinhamento** entre train e test: colunas ausentes no test são preenchidas com 0
4. **Sanitização** de nomes de colunas: caracteres especiais substituídos por _ (compatibilidade com LightGBM)
5. Salvamento em **formato Parquet** (columnar, comprimido)

O resultado final é um dataset com **1.042 features** e 307.511 linhas de treino.

5 Modelagem

5.1 Escolha do Algoritmo

O **LightGBM** (Light Gradient Boosting Machine) foi escolhido pelos seguintes motivos:

- **Eficiência com alta dimensionalidade:** os algoritmos GOSS e EFB permitem treinar rapidamente com mais de 1.000 features
- **Robustez a valores ausentes:** lida nativamente com NaN, aprendendo a melhor direção de divisão para dados ausentes

- **Performance em dados tabulares:** boosting de árvores é historicamente superior a redes neurais em dados tabulares com features de domínio
- **Suporte a desbalanceamento:** parâmetro `scale_pos_weight` ajusta o peso da classe minoritária no cálculo do gradiente

5.2 Otimização de Hiperparâmetros com Optuna

O Optuna implementa o **TPE** (*Tree-structured Parzen Estimator*), um método de otimização bayesiana que constrói um modelo probabilístico do espaço de hiperparâmetros e sugere configurações com maior probabilidade de melhoria.

O processo de tuning utilizou:

- 20 *trials*, cada um avaliado com 3-fold CV
- Métrica objetivo: AUC-ROC médio dos 3 folds
- Early stopping de 30 rodadas por fold para acelerar a busca

Hiperparâmetro	Espaço de Busca	Tipo
`learning_rate`	[0.02, 0.15]	log-uniform
`num_leaves`	[20, 200]	inteiro
`max_depth`	[4, 10]	inteiro
`min_child_samples`	[20, 300]	inteiro
`feature_fraction`	[0.4, 0.9]	uniforme
`bagging_fraction`	[0.4, 0.9]	uniforme
`bagging_freq`	[1, 7]	inteiro
`reg_alpha`	[10^{-4}, 5.0]	log-uniform
`reg_lambda`	[10^{-4}, 5.0]	log-uniform

Tabela 3: Espaço de busca do Optuna

5.3 Treinamento Final com 5-Fold CV

Após identificar os melhores hiperparâmetros, o modelo final é treinado com **StratifiedKFold** de 5 folds, que garante a preservação da proporção de inadimplência em cada fold:

Listing 3: Treinamento com validação cruzada estratificada

```
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
oof_preds  = np.zeros(len(X))
test_preds = np.zeros(len(X_test))

for fold, (tr_idx, val_idx) in enumerate(skf.split(X, y)):
    model = lgb.LGBMClassifier(**BEST_PARAMS)
    model.fit(X.iloc[tr_idx], y.iloc[tr_idx],
              eval_set=[(X.iloc[val_idx], y.iloc[val_idx])],
              callbacks=[lgb.early_stopping(100)])
    oof_preds[val_idx]  = model.predict_proba(X.iloc[val_idx])
   [:, 1]
```

```
    test_preds          += model.predict_proba(X_test)[:, 1] / 5
```

As predições de teste são a **média** das 5 predições, reduzindo variância.

5.4 Métrica OOF vs. Média dos Folds

O **AUC OOF** (Out-of-Fold) é calculado concatenando todas as predições out-of-fold e computando o AUC sobre o dataset completo:

$$\mathrm{AUC}_{\mathrm{OOF}} = \mathrm{AUC}(\mathbf{y}_{\mathrm{train}},\ \hat{\mathbf{p}}_{\mathrm{OOF}}) \tag{9}$$

Isso é diferente da média das AUCs por fold:

$$\overline{\mathrm{AUC}}_{\mathrm{folds}} = \frac{1}{K}\sum_{k=1}^{K} \mathrm{AUC}\Big(\mathbf{y}_{\mathrm{val}}^{(k)},\ \hat{\mathbf{p}}_{\mathrm{val}}^{(k)}\Big) \tag{10}$$

A AUC OOF é mais confiável pois avalia o **ranking global** de todas as predições juntas. Folds individuais podem ter AUC mais alta por dados estatisticamente mais separáveis — o OOF corrige esse viés.

5.5 Tracking com MLflow

Todos os experimentos são registrados no MLflow:

- Hiperparâmetros do melhor trial do Optuna
- AUC OOF final e AUC por fold
- Número de features utilizadas

6 Boas Práticas e Reprodutibilidade

6.1 Estrutura de Código

A separação entre notebooks e código reutilizável (`src/features.py`) evita duplicação. As funções `agg_numeric()` e `agg_categorical()` são genéricas e funcionam para qualquer tabela, recebendo apenas o DataFrame, a chave primária e o prefixo desejado.

6.2 Gerenciamento de Memória

Com datasets de dezenas de milhões de linhas, o uso de memória é crítico:

- **Downcast de tipos:** inteiros e floats convertidos para o menor tipo que comporta seus valores (`int8`, `int16`, `float32`)
- **Liberação explícita:** após cada tabela auxiliar, o objeto é deletado e `gc.collect()` é chamado
- **Parquet:** armazenamento intermediário columnar e comprimido

6.3 Prevenção de Data Leakage

O alinhamento de colunas entre train e test é feito via `pd.DataFrame.align(join='left')` para garantir que o test nunca possua features derivadas de informações do target. O Target Encoding, quando aplicado, usa K-Fold para evitar vazação.

7 Conclusão e Próximos Passos

7.1 Conclusão

Este projeto implementa um pipeline end-to-end robusto para predição de inadimplência. Os pontos técnicos principais são:

1. EDA orientada a decisões, não apenas descritiva
2. Engenharia de features fundamentada em domínio financeiro
3. Validação cruzada estratificada para lidar com desbalanceamento
4. Otimização bayesiana de hiperparâmetros com Optuna
5. Tracking completo de experimentos com MLflow

7.2 Próximos Passos

- **Stacking/Blending:** combinar LightGBM com XGBoost e CatBoost
- **Target Encoding:** para variáveis de alta cardinalidade como `ORGANIZATION_TYPE`
- **SHAP values:** seleção de features baseada em importância de permutação
- **Pseudo-labeling:** usar predições de alta confiança no test para ampliar o treino

A Dependências

Biblioteca	Versão
pandas	2.2.2
numpy	1.26.4
scikit-learn	1.4.2
lightgbm	4.x
optuna	3.6.1
mlflow	2.11.3
pyarrow	15.0.2
matplotlib	3.8.4
seaborn	0.13.2

Tabela 4: Dependências principais do projeto